Parallel and Distributed K-Nearest-Neighbor Classification: Comparing POSIX Threads, OpenMP, and MPI

Rahat Rizvi Rahman
Department of Computer Science
Virginia Commonwealth University, VCU
Richmond, USA
rahmanr12@vcu.edu

***Abstract*—This report presents a serial baseline and three parallel implementations of the k-nearest-neighbor (KNN) classifier: a shared-memory version using POSIX threads, a shared-memory version using OpenMP, and a distributed version using MPI. All three parallel versions partition the classification work along the same axis, the set of test instances, so that every worker reproduces the exact predictions of the serial version regardless of how many workers are used, which was confirmed exactly on every one of 244 completed runs across two clusters. The implementations were developed and correctness tested on `maple.cs.vcu.edu`, then evaluated for runtime and speedup on `maple.cs.vcu.edu` and, in place of an inaccessible `athena.hprc.vcu.edu` account, on the `hickory.cs.vcu.edu` cluster, using three datasets of increasing size. On hickory's dedicated, uncontended allocation all three implementations scale consistently with worker count, reaching up to a 5.8× speedup at 6 workers on the largest dataset, while identical code on the shared, unscheduled maple machine shows only modest gains on the larger datasets and measurable regressions on the smallest one, isolating scheduling and contention, rather than the parallelization approach itself, as the deciding factor.**

***Index Terms*—HPDS, KNN, POSIX threads, OpenMP, MPI, parallel computing, speedup**

I. Introduction

The k-nearest-neighbor classifier assigns a class to a query instance by finding the k closest instances to it in a labeled training set and taking a majority vote of their labels. Its accuracy is competitive with far more complex classifiers on many problems, but its cost grows as $O(num_test \times num_train \times num_features)$, since every test instance must be compared against every training instance across every feature. On datasets with tens of thousands of instances this cost becomes the practical limit on how quickly the classifier can be run, which makes KNN a useful vehicle for comparing parallelization strategies: the algorithm itself is simple enough that the effect of the parallelization strategy, rather than the complexity of the algorithm, dominates the results.

This report implements and evaluates three parallel strategies for the same classification workload. All three are built around the same partitioning decision, splitting the test instances across workers rather than the training instances, for a reason argued in Section II-B: it is the only one of the two splits that reproduces the serial version's predictions exactly, including on ties, without any extra work.

The rest of this report is organized as follows. Section II describes the serial algorithm, the shared partitioning strategy, the three parallel implementations, and the experimental setup and metrics used to evaluate them. Section III presents the measured runtimes, speedups, and accuracy. Section IV discusses the trends in those results and compares the three approaches. Section V concludes.

II. Methodology

A. Serial KNN Algorithm

The provided serial implementation classifies each test instance independently. For a single test instance, it computes the Euclidean distance to every training instance, maintains a sorted list of the k closest training instances seen so far, and after scanning the full training set predicts the class that appears most often among those k neighbors. A new training instance is inserted into the candidate list only when its distance is strictly smaller than an existing candidate's distance, so among instances at exactly equal distance, the one encountered earlier in the scan keeps its place. This tie-breaking rule matters for Section II-B.

B. Work Partitioning Strategy

All three parallel implementations divide the set of test instances into contiguous blocks and assign one block to each worker (thread or MPI process), which then scans the entire training set for each test instance in its block, exactly as the serial version does. Worker t of P workers is given the half-open range $[\lfloor tn/P \rfloor, \lfloor (t+1)n/P \rfloor)$ over n test instances. This splits the test instances into P blocks whose sizes differ by at most one, and it degrades gracefully when P exceeds n: a worker with $t \geq n$ receives an empty range and simply does no work, which occurs in this evaluation whenever the small dataset (160 test instances) is run with more than 160 workers.

An alternative partitioning splits the training set instead, and has every worker scan a shard of the training instances for the same query, building partial candidate lists that are then

merged. This exposes more parallel work, since the training sets used here are an order of magnitude larger than the test sets, but it changes the order in which training instances are compared for a given query. Because the serial algorithm breaks distance ties by scan order, merging partial candidate lists in a different order than the original single-threaded scan can change which neighbor is kept when two training instances are exactly equidistant from a query, changing the resulting prediction. Splitting on the test set instead preserves the original scan order inside every worker, so every implementation in this report reproduces the serial predictions exactly. This is confirmed empirically in Section III.

Splitting on the test set is also already load balanced without any runtime scheduling. Every test instance costs exactly $num_train \times num_attributes$ operations to classify, so an equal-sized static split already assigns equal work to every worker, and none of the three implementations uses a dynamic work-stealing or work-queue scheme.

C. POSIX Threads Implementation

The pthreads version packs every value a worker thread needs, the shared read-only dataset pointers, the worker's assigned range of test instances, and a pointer to the shared output array, into a struct, and allocates one struct instance per thread so that no two threads ever read or write the same struct while a thread might still be starting up. Each worker writes predictions only into the slice of the output array given by its own range, so no lock is needed on the output array, since the ranges assigned to different threads never overlap.

Each worker also allocates its own private copy of the two scratch buffers used during classification, the sorted candidate list and the per-class vote counts, once when the thread starts, and reuses that private copy across every test instance in its range. The serial version reuses a single copy of these buffers across all test instances because only one instance is ever being classified at a time; sharing that same buffer between multiple threads would let two threads overwrite each other's in-progress candidate list, so each thread is given its own independent copy instead. The main thread waits for every worker thread to finish with `pthread_join` before reading the completed predictions array, which acts as the barrier that guarantees every write is visible before the result is used.

D. OpenMP Implementation

The OpenMP version expresses the same partitioning using `#pragma omp parallel` to start a team of threads, followed by a separate `#pragma omp for`, with no `schedule` clause, to divide the test-instance loop across that team. The two pragmas are kept separate, rather than using the combined `parallel for` form, so that each thread's private scratch buffers can be allocated once, between the two pragmas, instead of once per test instance.

Those scratch buffers are declared as ordinary local variables inside the parallel region rather than passed through OpenMP's `private` or `firstprivate` data-sharing clauses. For a pointer variable, both clauses only copy the pointer's value, an address, not the buffer that address points to; every thread would therefore end up pointing at the same shared buffer, or in the case of `private`, at an uninitialized address. Declaring the buffers inside the parallel region instead relies on the fact that every thread executing that region runs the declaration itself and so receives its own independent storage, without needing any explicit data-sharing clause. No `schedule` clause is written because, as argued in Section II-B, every test instance costs the same amount of work, so the fixed, contiguous split every mainstream OpenMP runtime defaults to is already balanced, and an explicit dynamic schedule would only add scheduling overhead for no benefit here. The implicit barrier at the end of the `omp for` construct guarantees every thread has finished writing its predictions before its scratch buffers are freed and before the parallel region closes.

E. MPI Implementation

Unlike the two shared-memory versions, MPI ranks are separate processes with separate address spaces, so data cannot be shared through a common pointer. The implementation avoids communicating the dataset itself by having every rank independently parse both the training and test files before the timed region begins, so every rank ends up with its own complete copy of the data without any dataset bytes crossing between processes. This trades per-rank memory for simplicity and for eliminating one whole category of communication, at the cost of not scaling to datasets too large to fit in a single node's memory, which is not a limitation for the datasets used here.

Every rank computes the same partition table used by the threaded version, covering every rank's range, not only its own, since the collection step described next needs the full table. Each rank then classifies its own range of test instances into a local array sized to exactly its own share of the work. Because each rank's scratch buffers and local results array exist only in that rank's own memory, there is no data race to avoid here the way there is with threads.

The one step with no equivalent in the threaded versions is collecting every rank's local results back into a single array on rank 0, which alone computes the confusion matrix. This is done with `MPI_Gatherv` rather than the simpler `MPI_Gather`, because `MPI_Gather` requires every rank to contribute the same number of elements, while the partition table gives blocks that differ by one whenever the test set does not divide evenly across ranks. `MPI_Gatherv` takes a per-rank count and a per-rank displacement, so each rank's contribution lands at the correct global offset regardless of block size, and rank 0 recovers the predictions in their original order. `MPI_Gatherv` is also a synchronization point, so an `MPI_Barrier` is placed immediately before the timer starts, to prevent a rank that happens to finish parsing its input files earlier than the others from including that head start, and the resulting idle wait inside `MPI_Gatherv`, in its own measured runtime.

F. Experimental Setup

Development, debugging, and correctness testing were done on `maple.cs.vcu.edu`, a single shared, unscheduled node with 28 logical CPUs used concurrently by other students, with no reservation mechanism. Full timing experiments were intended to run on both `maple.cs.vcu.edu` and the `athena.hprc.vcu.edu` Slurm cluster, with the MPI implementation run twice on Athena, once with every rank forced onto a single node and once allowed to spread across multiple nodes, to compare intra-node against inter-node communication cost. This plan could not be carried out: the assigned Athena account returned access errors for the full duration of the assignment period despite being reported to the course staff, so no Athena results appear in this report.

In its place, the `hickory.cs.vcu.edu` cluster, a separate VCU HPRC system, was used for the reserved-allocation comparison against maple. hickory imposes constraints that do not exist on Athena and that limit what could be measured there. Its Slurm configuration exposes a single node in a single, GPU-only partition, so there is no second node to place a multi-node MPI run on; the single-node-versus-multi-node MPI comparison the assignment specifies for Athena could therefore not be performed on hickory, and only the single-node MPI case is reported. The partition's job-submission policy also rejects any job that requests no GPU resource, so every hickory job requests one unit of the smallest available GPU type purely to satisfy that policy, without the benchmark ever touching it. The account's QOS caps a single job at 6 CPUs, so hickory worker counts are limited to 1, 2, 4, and 6, rather than the full 1–128 range used on maple.

Three datasets of increasing size were used, summarized in Table I. Each was evaluated with $k = 3$. On maple, worker counts of 1, 2, 4, 8, 16, 32, 64, and 128 were attempted; oversubscribing maple's 28 physical CPUs with dozens to well over a hundred threads or processes, on a machine already shared with other students, made the higher worker counts on the two larger datasets take impractically long (single configurations were observed spending over 95% of their wall-clock time waiting for a CPU rather than computing), so the maple results reported here cover the small and medium datasets at worker counts up to 64 (32 for MPI); the large dataset completed only its serial baseline on maple before this became impractical. hickory's own worker counts are the 1, 2, 4, 6 range its CPU cap allows. Every configuration was run three times and the runtimes reported in Section III are the mean of those three runs.

TABLE I
DATASET SIZES

Dataset	Train instances	Test instances	Attributes
small	1,184	160	7
medium	14,708	740	11
large	61,606	3,436	11

G. Performance Metrics

Two speedup definitions are reported, since they answer different questions. Absolute speedup compares a parallel run against the serial program,

$$S_{\text{abs}}(p) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(p)}, \tag{1}$$

and reflects the real-world benefit of parallelizing, including whatever fixed overhead the parallel version pays. Relative speedup instead compares a parallel implementation against its own single-worker run,

$$S_{\text{rel}}(p) = \frac{T_{\text{parallel}}(1)}{T_{\text{parallel}}(p)}, \tag{2}$$

which isolates how well that implementation scales once its own fixed overhead is factored out. Absolute speedup is used as the headline metric in Section III, with relative speedup reported alongside it.

The parallel fraction of the code is estimated from the absolute speedup using the Karp-Flatt metric,

$$e(p) = \frac{1/S_{\text{abs}}(p) - 1/p}{1 - 1/p}, \qquad f_{\text{parallel}}(p) = 1 - e(p), \tag{3}$$

where $e(p)$ is the estimated serial fraction at p workers. Unlike Amdahl's law solved for a single fixed serial fraction, Karp-Flatt is evaluated at every measured value of p; an estimated serial fraction that grows as p increases is itself evidence of parallel overhead, such as load imbalance or communication cost, that a model with one fixed serial fraction cannot capture.

III. RESULTS

A. Accuracy

Table II reports the classification accuracy of the serial implementation on each dataset. Every parallel implementation, at every worker count tested on either cluster, reproduced these accuracies exactly, confirmed with an automated check comparing each parallel run's reported accuracy against the serial run's accuracy for the same dataset; across all 244 completed runs on maple and hickory combined, not one mismatch was observed. The lower accuracy on the medium dataset is a property of that dataset's class separability, not a defect in any implementation, since it is reproduced identically by the serial version and by every parallel run.

TABLE II
SERIAL CLASSIFICATION ACCURACY, $k = 3$

Dataset	Accuracy
small	85.00%
medium	27.57%
large	99.48%

B. Runtime and Speedup on Maple

Tables III–V report mean runtime, absolute speedup, and estimated parallel fraction for all three implementations on maple, for the small and medium datasets; as explained in Section II, the large dataset did not finish beyond its serial baseline on maple within a practical amount of time and is omitted from these tables. Figure 1 plots absolute speedup against worker count for all three implementations on the medium dataset, the largest one with complete maple coverage.

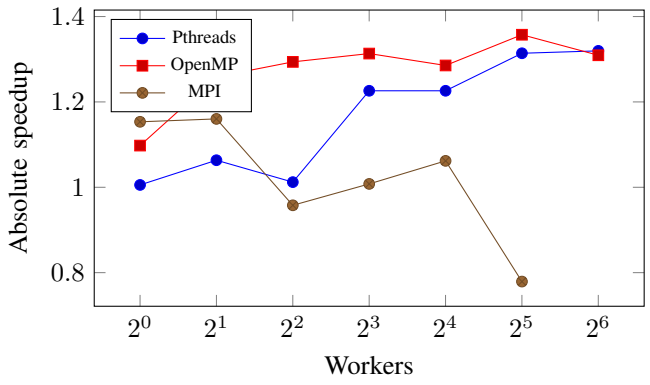


Fig. 1. Absolute speedup vs. worker count on maple, medium dataset.

C. Runtime and Speedup on hickory

Tables VI–VIII report mean runtime, absolute speedup, and estimated parallel fraction for all three implementations on hickory, across all three datasets and the worker counts hickory's 6-CPU job cap allows. Table VIII reports only the single-node case, since hickory has no second node to place a multi-node MPI run on, so the single-node-versus-multi-node comparison Section II describes for Athena does not appear here. Figure 2 plots absolute speedup against worker count for all three implementations on the large dataset.

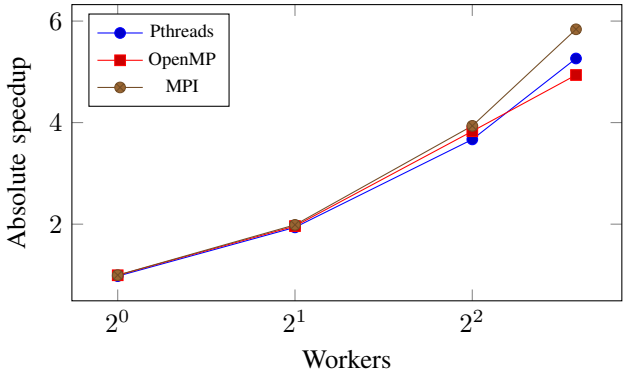


Fig. 2. Absolute speedup vs. worker count on hickory, large dataset.

IV. Analysis and Discussion

The two clusters tell opposite stories from the same code, and the difference is attributable to resource scheduling rather than to any implementation. On hickory, every job receives a dedicated Slurm allocation that no other job touches, and Tables VI–VIII show the consequence directly: all three implementations scale consistently with worker count on the medium and large datasets, reaching between 4.9× and 5.8× speedup at 6 workers on the large dataset, with an estimated parallel fraction above 0.9 for nearly every medium and large configuration. That high parallel fraction matches the algorithm's structure, since the only serial work in any implementation is the single confusion-matrix pass over the finished predictions, an $O(num_classes^2)$ operation that is negligible next to the $O(num_test \times num_train \times num_attributes)$ distance computation every worker performs in parallel.

Maple's shared, unscheduled hardware shows a materially different picture on the same algorithm. On the medium dataset, all three implementations still show real but far more modest gains, topping out around 1.3×–1.4× speedup for pthreads and OpenMP, well short of hickory's numbers at similar worker counts. MPI on maple's medium dataset is worse still: speedup is roughly flat through 16 workers and then drops below 1 at 32 workers (Table V), meaning 32 concurrent MPI ranks finished slower than one. MPI ranks are heavier OS entities than threads, each with its own scheduling footprint and its own participation in the `MPI_Barrier` and `MPI_Gatherv` synchronization points described in Section II; on a machine already shared with other users' processes, oversubscribing it with 32 such ranks multiplies contention for the same physical cores far more than 32 threads inside one process would, and every synchronization point then waits on whichever rank the scheduler starved the longest.

On the small dataset this effect is severe enough to produce negative estimated parallel fractions in every maple table, for example OpenMP at 64 workers (Table IV) taking 20.0 ms against a 9.3 ms serial baseline, slower with 64 workers than with one. Small's runs complete in only a few milliseconds, so the fixed cost of creating threads or ranks and competing with other users' processes for scarce cores dominates a workload that is barely there to begin with; hickory shows traces of the same effect at its own highest worker count on small (Tables VI–VII), just far less severely, since a dedicated allocation still removes the cross-user-contention half of the problem. Karp-Flatt is doing its job correctly in both cases: a negative estimated parallel fraction is not a nonsensical result but an honest signal that overhead, not serial code, is what is limiting speedup at that configuration.

Comparing the three implementations directly on hickory's large dataset at 6 workers, the highest worker count tested, MPI reaches 5.84×, pthreads 5.26×, and OpenMP 4.94×. The three are within roughly 15% of each other, consistent with all three performing the same per-worker distance computation once started; the gap is not large enough, relative to the three repetitions each figure is already averaged over, to claim a definitive ranking between them, only that none of the three pays a heavy implementation-specific tax on hickory's dedicated hardware.

Two comparisons the assignment specifies could not be made in this report. The single-node-versus-multi-node MPI

TABLE III
PTHREADS ON MAPLE ($k = 3$)

Workers	small			medium		
	Time (ms)	S_{abs}	f_{parallel}	Time (ms)	S_{abs}	f_{parallel}
1	10.0	0.93	—	722.7	1.01	—
2	8.0	1.17	0.29	683.3	1.06	0.12
4	8.0	1.17	0.19	718.0	1.01	0.02
8	10.3	0.90	-0.12	592.7	1.23	0.21
16	12.0	0.78	-0.30	592.7	1.23	0.20
32	17.0	0.55	-0.85	553.0	1.31	0.25
64	12.0	0.78	-0.29	550.7	1.32	0.25

TABLE IV
OPENMP ON MAPLE ($k = 3$)

Workers	small			medium		
	Time (ms)	S_{abs}	f_{parallel}	Time (ms)	S_{abs}	f_{parallel}
1	10.3	0.90	—	662.0	1.10	—
2	15.3	0.61	-1.29	577.3	1.26	0.41
4	6.7	1.40	0.38	561.7	1.29	0.30
8	13.7	0.68	-0.53	553.3	1.31	0.27
16	10.3	0.90	-0.11	565.3	1.29	0.24
32	14.3	0.65	-0.55	535.3	1.36	0.27
64	20.0	0.47	-1.16	555.0	1.31	0.24

TABLE V
MPI ON MAPLE ($k = 3$)

Workers	small			medium		
	Time (ms)	S_{abs}	f_{parallel}	Time (ms)	S_{abs}	f_{parallel}
1	6.7	1.40	—	630.0	1.15	—
2	5.0	1.87	0.93	626.3	1.16	0.28
4	12.3	0.76	-0.43	758.7	0.96	-0.06
8	18.3	0.51	-1.10	721.0	1.01	0.01
16	7.0	1.33	0.27	684.3	1.06	0.06
32	29.3	0.32	-2.21	932.7	0.78	-0.29

TABLE VI
PTHREADS ON HICKORY ($k = 3$)

Workers	small			medium			large		
	Time (ms)	S_{abs}	f_{parallel}	Time (ms)	S_{abs}	f_{parallel}	Time (ms)	S_{abs}	f_{parallel}
1	6.7	0.85	—	429.0	0.97	—	8286.0	0.98	—
2	3.0	1.89	0.94	236.0	1.77	0.87	4177.3	1.94	0.97
4	3.7	1.55	0.47	178.7	2.34	0.76	2204.7	3.67	0.97
6	2.7	2.12	0.64	171.3	2.44	0.71	1537.0	5.26	0.97

comparison has no hickory counterpart, since hickory exposes only one node; Table VIII reports the single-node case only, and no statement can be made here about inter-node communication cost. Athena itself, which would have supported that comparison, was inaccessible for the entire assignment period despite being reported to the course staff, so neither cluster's data in this report can speak to it. Separately, the worker-count ranges tested differ by cluster for two different reasons: hickory's ceiling of 6 is an administrative QOS limit, not a technical one, while maple's practical ceiling on the two larger datasets is the oversubscription effect described above and in Section II. Both are themselves findings about how scheduling policy and resource contention, not the parallelization code, bound what is achievable in practice.

V. CONCLUSION

This report implemented and evaluated three parallel strategies, POSIX threads, OpenMP, and MPI, for the k-nearest-neighbor classifier, all built around the same test-instance partitioning strategy. That shared strategy was chosen specifically because it reproduces the serial version's predictions exactly regardless of worker count, which was confirmed empirically

TABLE VII
OPENMP ON HICKORY ($k = 3$)

Workers	small			medium			large		
	Time (ms)	S_{abs}	f_{parallel}	Time (ms)	S_{abs}	f_{parallel}	Time (ms)	S_{abs}	f_{parallel}
1	6.0	0.94	—	420.7	0.99	—	8142.3	0.99	—
2	3.0	1.89	0.94	244.7	1.71	0.83	4122.7	1.96	0.98
4	10.7	0.53	-1.18	169.3	2.47	0.79	2112.0	3.83	0.99
6	15.3	0.37	-2.05	146.7	2.85	0.78	1638.3	4.94	0.96

TABLE VIII
MPI ON HICKORY, SINGLE NODE ($k = 3$)

Workers	small			medium			large		
	Time (ms)	S_{abs}	f_{parallel}	Time (ms)	S_{abs}	f_{parallel}	Time (ms)	S_{abs}	f_{parallel}
1	5.0	1.13	—	418.7	1.00	—	8121.7	1.00	—
2	2.7	2.12	1.06	212.0	1.97	0.99	4072.3	1.99	0.99
4	4.3	1.31	0.31	134.0	3.12	0.91	2055.0	3.94	0.99
6	3.0	1.89	0.56	92.3	4.53	0.93	1385.7	5.84	0.99

across all three implementations, three datasets, and all 244 completed runs on both clusters used in this study.

The results show that the same parallelization code can succeed or fail depending entirely on the hardware it runs on. On hickory's dedicated, uncontended Slurm allocation, all three implementations scaled consistently with worker count, reaching up to 5.8× speedup on the large dataset at 6 workers with an estimated parallel fraction above 0.9, matching KNN's embarrassingly parallel structure. On maple's shared, unscheduled hardware, the same code produced only modest gains on the medium dataset and measurable slowdowns on the small one, since the fixed cost of creating threads or MPI ranks, and competing with other users' processes for the same physical cores, outweighed the benefit of parallelizing workloads that were themselves only milliseconds long. Athena, which the assignment specifies for the single-node-versus-multi-node MPI comparison, was inaccessible for the entire assignment period despite being reported to the course staff; the hickory results substituted here are otherwise complete but, having only one physical node, cannot speak to that specific comparison.

REFERENCES

[1] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[2] B. Lewis and D. J. Berg, *Multithreaded Programming with Pthreads*. Prentice-Hall, Inc., 1998.

[3] OpenMP Architecture Review Board, "OpenMP Application Programming Interface, Version 5.2," 2021.

[4] W. Gropp, E. Lusk, and A. Skjellum, *Using MPI: Portable Parallel Programming with the Message-Passing Interface*, 3rd ed. MIT Press, 2014.

[5] A. H. Karp and H. P. Flatt, "Measuring parallel processor performance," *Communications of the ACM*, vol. 33, no. 5, pp. 539–543, 1990.